

StudyBuddy.io

AI-Powered Study Companion for Students

Capstone Project Documentation

Software Engineering Bootcamp

Prepared by Amanda McIntire

2026

Table of Contents

1. Project Overview	3
Problem Statement	
User Stories	
Wireframes	
2. Architecture Explanation	7
Frontend	
Backend	
Database	
External Integrations	
3. MongoDB Schema / Model Explanation	8
User	
Subject	
Note	
Flashcard	
Cheatsheet	
Quiz	
Rewrite	
PasswordResetCode	
ContactMessage	
4. AI-Powered Feature Explanation	11
Feature 1: Generate Quiz from Selected Notes	
Feature 2: Rewrite for Clarity	
Prompt Design Principles Applied	
5. Testing & Quality Assurance	12
Automated Testing (Jest)	
Manual End-to-End Testing	
6. Summary of AI Usage During Development	13
How AI Was Used	
What Was Not Simply Accepted	
Why This Matters for This Documentation	

1. Project Overview

StudyBuddy.io is a full-stack MERN (MongoDB, Express, React, Node.js) web application designed to help students organize their coursework and prepare for exams more effectively. The application was built as a capstone project for a Software Engineering bootcamp, with an explicit requirement to integrate a large language model (Claude, by Anthropic) as a core, purposeful feature — rather than as a superficial add-on.

The application allows a student to create Subjects (e.g., "Biology", "Chemistry"), and within each subject: take rich-text Notes, build Flashcards for quick review, assemble two-sided Cheat Sheets modeled on real exam "allowed notes" pages, and generate AI-powered practice Quizzes drawn directly from their own notes. Every account is protected by secure authentication, and every resource is strictly scoped to its owner.

Beyond the core study tools, the application includes account management (name, email, and password changes with password confirmation), a self-service password reset flow via emailed one-time codes, a dark mode / light mode theme system, a Help & FAQ page, and a Contact Us form for user feedback.

Problem Statement

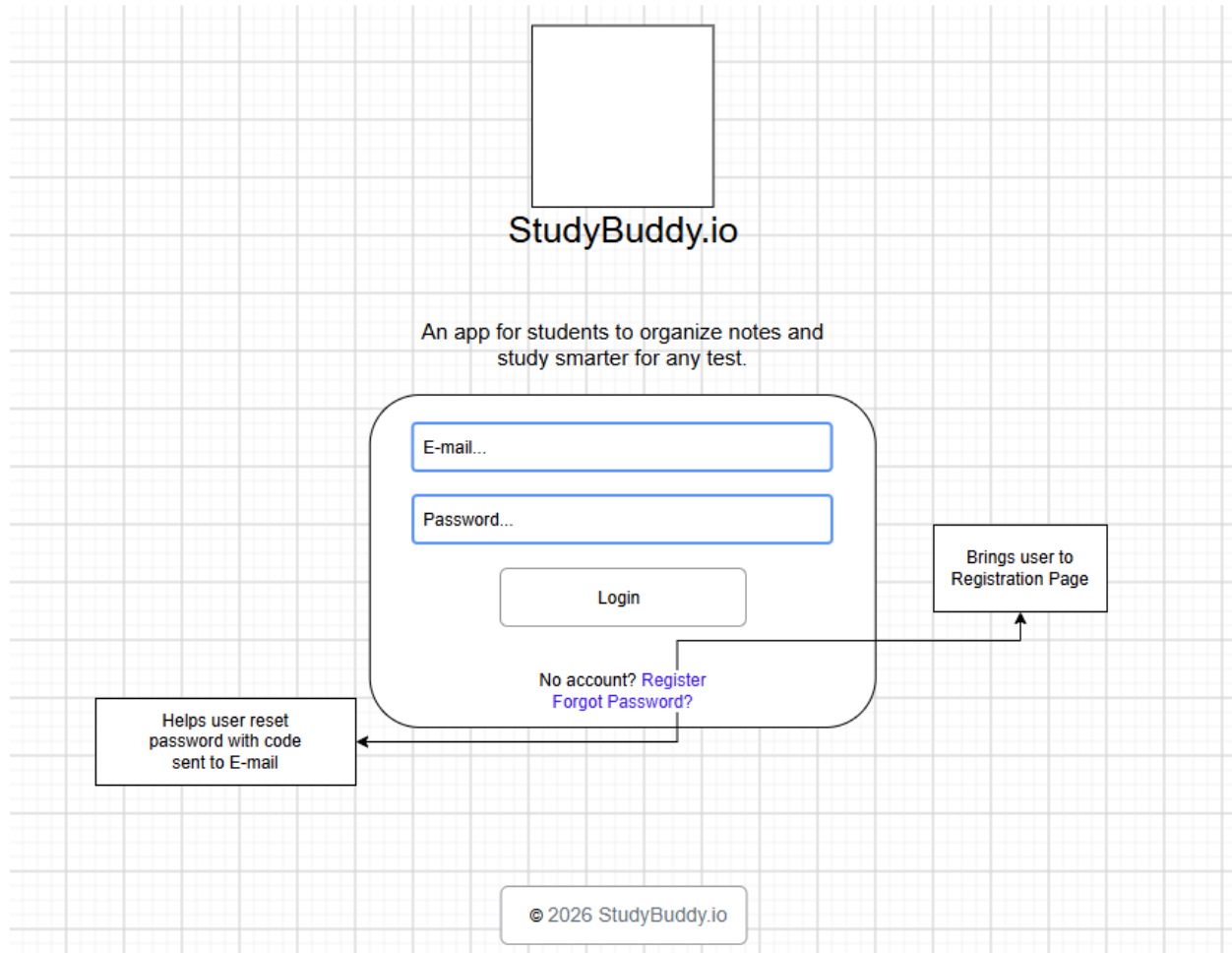
Students often juggle multiple disconnected tools for note-taking, flashcard review, and exam preparation, with no easy way to convert study material into active-recall practice. StudyBuddy.io consolidates these workflows into a single application and uses AI specifically to close the gap between "having notes" and "being tested on them" — turning passive material into active practice quizzes and clearer, more concise study content, with minimal manual effort from the student.

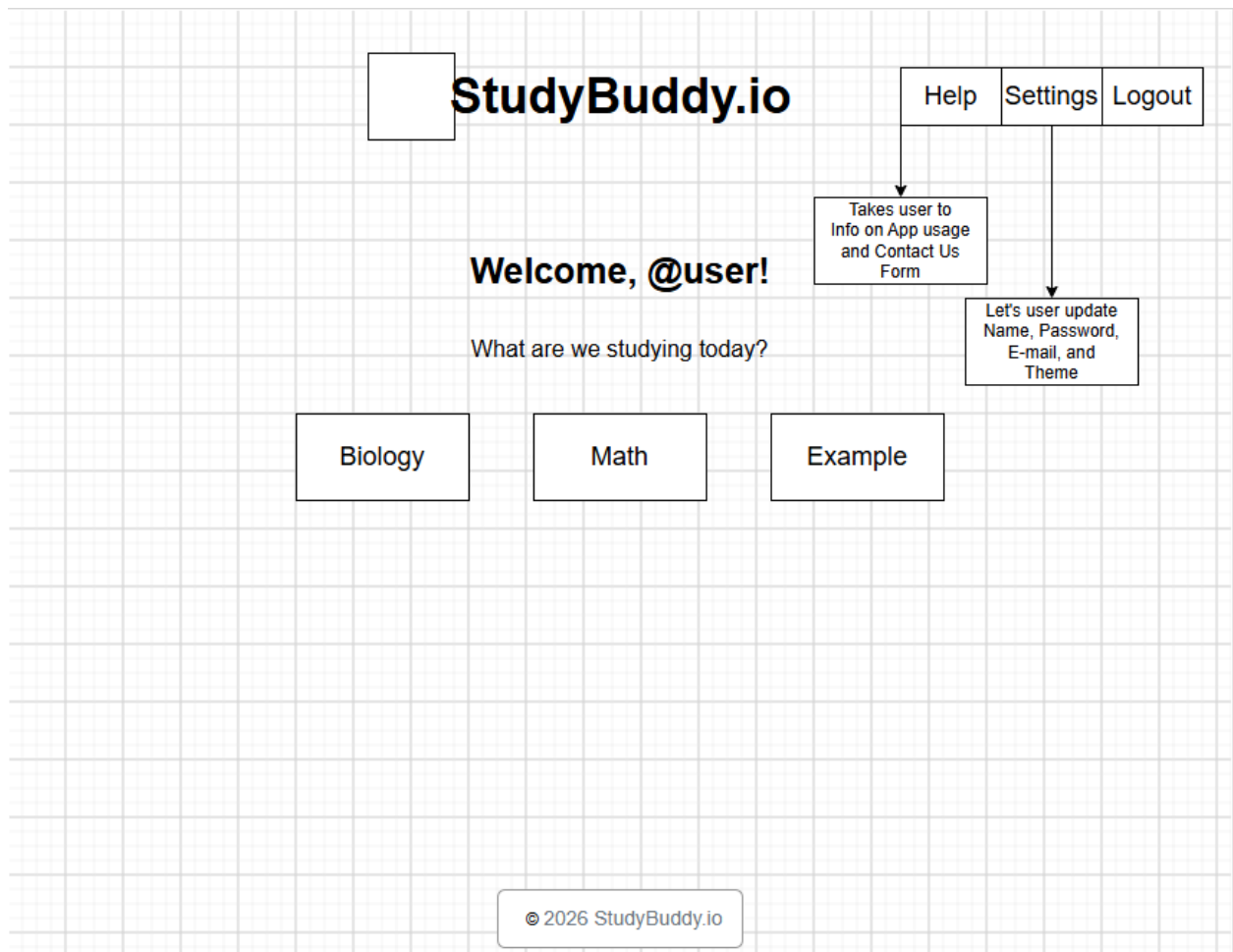
User Stories

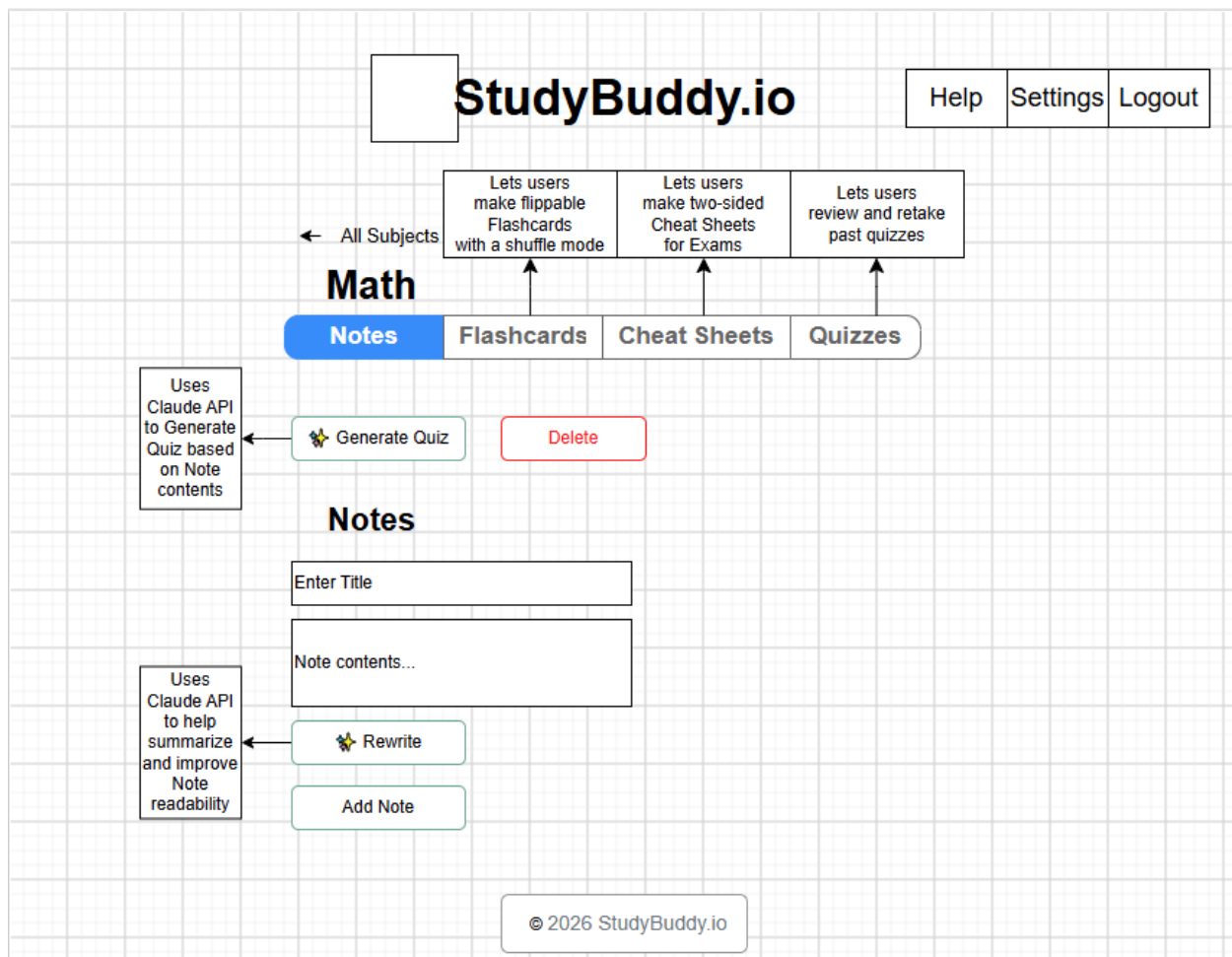
- As a user, I want to create and save notes for study and test preparation on a given subject.
- As a user, I want to organize notes into different categories and subjects.
- As a user, I want to create flash cards to review my knowledge on a given subject.
- As a user, I want to generate pop quizzes to test my knowledge on a given subject.
- As a user, I want my notes to be summarized for improved readability.
- As a user, I want to protect and save my notes in a secure account.

Wireframes

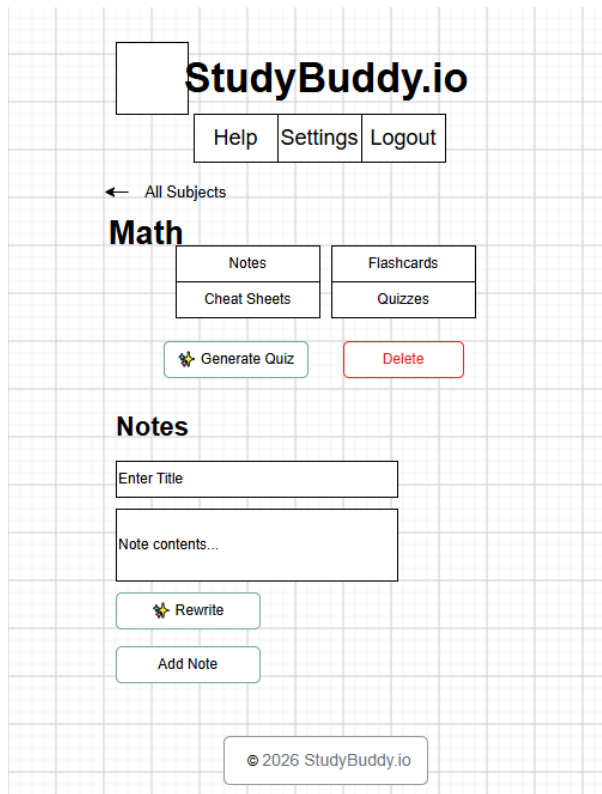
Login





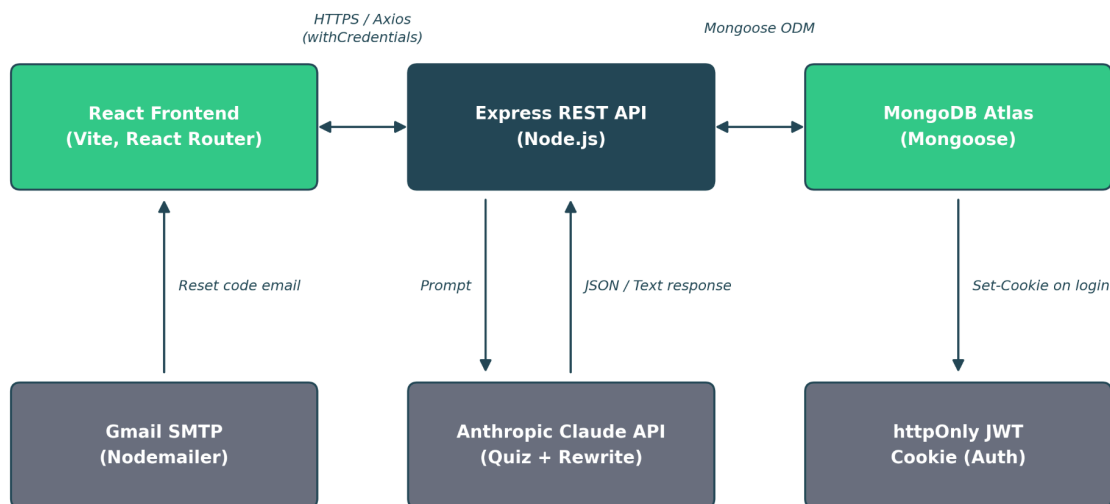


Subject (Mobile)



2. Architecture Explanation

StudyBuddy.io follows a standard three-tier MERN architecture, with a clearly separated frontend, backend, and database layer, plus two external service integrations (Anthropic's Claude API and Gmail's SMTP service via Nodemailer).



Frontend

Built with React 19 and Vite. React Router handles client-side navigation across public routes (Login, Register, Forgot Password) and protected routes (Dashboard and every subject-scoped page), gated by a ProtectedRoute component that checks authentication state before rendering. Axios is configured with `withCredentials: true` so the browser automatically sends the `httpOnly` authentication cookie on every request, without any manual token handling in application code.

Backend

Built with Node.js and Express, structured around RESTful resource routes (auth, subjects, notes, flashcards, cheatsheets, quizzes, rewrites, contact). Authentication is handled via JSON Web Tokens stored in `httpOnly`, `sameSite` cookies — never in `localStorage` — which protects the token from theft via cross-site scripting (XSS). Every protected route runs through a shared `requireAuth` middleware that verifies the token and attaches the authenticated user's ID to the request, and every resource query is filtered by that owner ID at the database level, not just hidden in the UI.

Database

MongoDB, hosted on MongoDB Atlas, accessed via Mongoose for schema definition and validation. Relationships between collections (e.g., a Note belonging to a Subject and a User) are modeled using Mongoose `ObjectId` references rather than a relational join, consistent with MongoDB's document-oriented design.

External Integrations

- Anthropic Claude API — powers both the quiz generation and note/cheat-sheet rewriting features, called exclusively from the backend so the API key is never exposed to the browser.
- Gmail SMTP (via Nodemailer) — sends one-time password reset codes as part of the Forgot Password flow.

3. MongoDB Schema / Model Explanation

The application uses nine Mongoose models. With the exception of the join-style relationship fields, every collection that stores user-specific data includes an owner field referencing the User model, which is the mechanism enforcing that users can only ever access their own data.

User

Field	Type / Description	Notes
name	String, required, max 30 chars	Display name
email	String, required, unique, lowercase	Used for login and password reset
password	String, required	bcrypt hash only — plaintext is never stored

Subject

Field	Type / Description	Notes
title	String, required, max 30 chars	
owner	ObjectId → User, required	Enforces per-user data isolation

Note

Field	Type / Description	Notes
title	String, required, max 30 chars	
body	String, required	Stores Quill-generated HTML (rich text)
subject	ObjectId → Subject, required	
owner	ObjectId → User, required	

Flashcard

Field	Type / Description	Notes
front / back	String, required, max 100 chars each	
subject	ObjectId → Subject, required	Max 15 flashcards enforced per subject
owner	ObjectId → User, required	

Cheatsheet

Field	Type / Description	Notes
title	String, required, max 30 chars	
frontContent / backContent	String	Quill HTML for each side of the sheet
subject	ObjectId → Subject, required	Max 5 cheat sheets enforced per subject
owner	ObjectId → User, required	

Quiz

Field	Type / Description	Notes
questions	Array of { question, options[4], correctIndex, explanation }	Structured output parsed from Claude's JSON response
subject	ObjectId → Subject, required	
owner	ObjectId → User, required	

Rewrite

Field	Type / Description	Notes
note / cheatsheet	ObjectId (one or the other set)	Rewrite can apply to either source type
originalText / rewrittenText	String	Plain-text before/after, for the comparison preview
owner	ObjectId → User, required	

PasswordResetCode

Field	Type / Description	Notes
codeHash	String, required	bcrypt hash of the 6-digit code — never stored in plaintext
expiresAt	Date, required	15-minute expiration window
attempts	Number, default 0	Caps brute-force guessing at 5 attempts
user	ObjectId → User, required	

ContactMessage

Field	Type / Description	Notes
name / email	String, required	Snapshoted at submission time
message	String, required	
user	ObjectId → User, required	

4. AI-Powered Feature Explanation

The Claude API is integrated in two distinct, purpose-built features. Both share the same Anthropic SDK client, API key handling, and error-handling pattern (missing key, rate limiting, malformed responses), implemented entirely on the backend.

Feature 1: Generate Quiz from Selected Notes

Input: One or more notes selected by the user via checkboxes.

Process: The selected notes' HTML content is stripped to plain text (with HTML entity decoding to avoid artifacts like ' appearing literally) and combined into a single block. This is sent to Claude with a prompt that explicitly requests strict JSON output — five multiple-choice questions, each with four options, a zero-based correct-answer index, and an explanation — with instructions to base every question only on the provided material.

Output: The parsed JSON is validated, persisted to MongoDB as a Quiz document, and rendered as an interactive quiz. Answered questions are color-coded (green for correct, red for incorrect) with explanations shown after submission, and past quizzes can be retaken or reviewed (answers-only view) from a dedicated Quizzes tab.

Feature 2: Rewrite for Clarity

Input: A single note, or one side (front/back) of a cheat sheet.

Process: The existing content is stripped to plain text and sent to Claude with instructions to rewrite it more clearly and concisely while explicitly preserving every fact and not introducing new information.

Output: The rewritten text is shown in a side-by-side before/after preview. The user can Apply (overwriting the original, wrapped back into valid HTML for the rich-text editor) or Discard.

Prompt Design Principles Applied

- Explicit output format instructions ("respond with ONLY valid JSON, no markdown fences") to make responses reliably parseable.
- Defensive parsing (stripping stray markdown fences, try/catch around JSON.parse) since LLM output format compliance is a best-effort, not a guarantee.
- Instructions to stay strictly grounded in the provided material, reducing the risk of the model introducing outside information not present in the student's own notes.
- A visible disclaimer: "AI can get things wrong. Always check the facts through verifiable sources."

5. Testing & Quality Assurance

Automated Testing (Jest)

Backend API routes are covered by an automated test suite using Jest, Supertest, and mongodb-memory-server (a real, temporary MongoDB instance created fresh for each test run, so tests never touch production data).

- `auth.test.js` — registration validation (password length, duplicate email rejection), login success/failure.
- `subjects.test.js` — CRUD operations, 30-character title limit, cascade deletion, and cross-user ownership isolation.
- `notes.test.js` — CRUD operations, validation, bulk deletion, and cross-user ownership isolation.

The ownership-isolation tests are the most significant from a security standpoint: they directly verify that one user can never read, edit, or delete another user's data — the core guarantee the entire authentication and authorization model is built to provide. All 18 tests pass, run via `npm test` in the backend directory.

Manual End-to-End Testing

In addition to automated backend tests, every feature in this application was manually end-to-end tested through the full browser-based user flow during development — registration through logout, creation through deletion, and every AI feature — rather than only being verified in isolation. This included deliberately testing failure paths, not just the happy path:

- Authentication: registration with invalid/duplicate data, session persistence across refresh, protected-route redirection when logged out.
- AI features: missing API key handling, malformed AI response handling, and rate-limit responses, in addition to successful generation.
- Password reset: incorrect codes, expired codes, and the full happy-path flow via a real emailed code.
- Cross-browser rendering checks and a dedicated accessibility pass (keyboard navigation, screen-reader label associations, color contrast) across every page.

This manual end-to-end process surfaced and resolved a number of real issues during development — including a Mongoose middleware version incompatibility, CORS misconfiguration, HTML entity encoding artifacts, and several CSS flexbox layout bugs — before they could reach a deployed environment.

6. Summary of AI Usage During Development

Claude (Anthropic) was used throughout this project's development as a collaborative pair-programming and tutoring tool, guiding an iterative, feature-by-feature build rather than generating the application in a single pass. This section documents that process transparently.

How AI Was Used

- Project scoping — narrowing an initial multi-feature concept down to a single, well-defined AI feature appropriate for the assignment's requirements, then expanding deliberately from there.
- Iterative code generation — building authentication, CRUD features, and AI integrations one at a time, with each piece tested and confirmed working before moving to the next.
- Debugging real issues — including CORS errors, environment variable misconfiguration, a Mongoose pre-save hook incompatibility, Gmail SMTP authentication failures, and multiple non-obvious CSS layout bugs (flexbox min-width behavior, scrollbar-induced layout shift, non-breaking space characters blocking text wrapping).
- Security guidance — httpOnly cookie-based authentication, per-request ownership checks, password reset token design, and brute-force attempt limiting.
- Accessibility review — identifying and fixing concrete WCAG issues (insufficient button contrast, missing form labels, keyboard-inaccessible custom controls) rather than a generic accessibility pass.
- Test suite construction — designing and writing the Jest/Supertest backend test suite, including the ownership-isolation tests.

What Was Not Simply Accepted

Every piece of AI-generated code was reviewed, manually tested in the running application, and — in several documented cases — debugged and corrected through iterative back-and-forth when it did not work as expected on the first attempt. This document itself, and the development conversation it is drawn from, reflects that iterative and critical process rather than a single generated output accepted at face value.

Why This Matters for This Documentation

Beyond fulfilling the assignment's requirement to use the Claude API as a feature, this project treats AI collaboration honestly as part of the development methodology — a practice increasingly relevant to how software is actually built in industry, and one this documentation aims to represent accurately rather than understate or overstate.